

自然语言处理核心任务与应用

一、NLP 基础知识

自然语言处理核心任务与应用

一、NLP 基础知识

1.1 Tokenization (分词 / 令牌化)

Tokenization 是 NLP 的第一步，将原始文本转换为模型可处理的基本单元 (token)。不同模型的词汇量、处理效率和语义理解能力。

1.1.1 为什么 Tokenization 如此重要？

- 词汇量控制：好的分词方法能在覆盖度和效率之间取得平衡
- 未登录词 (OOV) 问题：分词策略决定了模型处理未见词汇的能力
- 多语言支持：不同语言需要不同的分词策略 (中文需要分词，英文天然有空格)
- 子词信息：子词分词能保留词缀信息 (如 "un-", "-ing", "-ed")

1.1.2 BPE (Byte Pair Encoding) 算法详解

BPE 是目前最流行的子词分词算法，GPT-2、GPT-3 等模型均使用 BPE。

核心思想：从字符级别开始，逐步合并最频繁的字符对，直到达到预设的词汇量。

算法步骤：

1. 将语料库中所有单词拆分为字符序列，并在末尾添加特殊的词尾标记 `< /w >`
2. 统计所有相邻字符对的频率
3. 将频率最高的字符对合并为一个新符号
4. 重复步骤 2 - 3，直到达到预设词汇量大小

详细示例：

假设语料库为：low lower lowest，统计词频为：low: 5, lower: 2, lowest: 1

步骤 1：拆分为字符

l o w < /w > : 5

```
l o w e r < / w > : 2
l o w e s t < / w > : 3
```

步骤2：统计字符对频率

```
( l , o ) : 5 + 2 + 3 = 10
( o , w ) : 5 + 2 + 3 = 10
( w , < / w > ) : 5
( w , e ) : 2 + 3 = 5
( e , r ) : 2
( r , < / w > ) : 2
( e , s ) : 3
( s , t ) : 3
( t , < / w > ) : 3
```

步骤3：合并频率最高的字符对 (l , o)

```
l o w < / w > : 5
l o w e r < / w > : 2
l o w e s t < / w > : 3
```

步骤4：继续合并 (l o , w)

```
l o w < / w > : 5
l o w e r < / w > : 2
l o w e s t < / w > : 3
```

继续... 最终可能得到词汇：low, low</w>, lower, lowest

BPE的Python实现：

```
import re
from collections import Counter, defaultdict

def getstats(vocab):
    """统计相邻符号对的频率"""
    pairs = Counter()
```

```

for word, freq in vocab.items():
    symbols = word.split()
    for i in range(len(symbols) - 1):
        pairs[(symbols[i], symbols[i+1])] += freq
    return pairs

def mergevocab(pair, vocab):
    """合并指定的字符对"""
    newvocab = {}
    # 构造正则表达式，匹配字符对（注意处理空格边界）
    bigram = re.escape(' '.join(pair))
    p = re.compile(r'(?!\S)' + bigram + r'(?!\S)')
    for word in vocab:
        # 将匹配的字符对合并
        newword = p.sub(' '.join(pair), word)
        newvocab[newword] = vocab[word]
    return newvocab

```

初始词汇表：每个字符用空格分隔，< /w>表示词尾

```

vocab = {
    'l o w </w>': 5,
    'l o w e r </w>': 2,
    'l o w e s t </w>': 3,
}

nummerges = 10 # 合并次数
for i in range(nummerges):
    pairs = getstats(vocab)
    if not pairs:
        break
    best = max(pairs, key=pairs.get)
    vocab = mergevocab(best, vocab)
    print(f"合并 {i+1}: {best} -> {' '.join(best)}, 频率: ")

print("\n最终词汇表:")

```

```
for word, freq in vocab.items():
    print(f" {word}: {freq} ")
```

1.1.3 WordPiece算法

WordPiece与BPE类似，但合并标准不同：BPE选择频率最高的对，WordPiece选择使语料对。

关键区别：

$$\text{Score}(A, B) = \frac{P(AB)}{P(A) \times P(B)}$$

WordPiece选择使上述得分最高的字符对进行合并，而非简单频率最高的对。这意味着它更倾向于合并在一起出现概率远高于独立出现概率"的字符对，从而更好地捕获有意义的子词单元。

BERT使用WordPiece分词，词汇量约30000。

1.1.4 SentencePiece

SentencePiece是一个语言无关的子词分词工具，被ALBERT、T5等模型采用。

特点：

- 将文本视为原始字节流，不需要预处理（如分词）
- 支持BPE和Unigram两种分词模式
- 直接从原始文本训练，无需预分词
- 支持多语言混合文本

```
import sentencepiece as spm
```

训练SentencePiece模型

```
spm.SentencePieceTrainer.train(
    input='corpus.txt',
    model_prefix='my_model',
    vocab_size=32000,
    model_type='bpe', # 或 'unigram'
)
```

使用模型分词

```

sp = spm.SentencePieceProcessor()
sp.load('mymodel.model')

text = "自然语言处理是人工智能的重要方向"
tokens = sp.encode_as_pieces(text)
ids = sp.encode_as_ids(text)
print(f"分词结果: {tokens}")
print(f"ID序列: {ids}")

```

解码

```

decoded = sp.decode_pieces(tokens)
print(f"解码结果: {decoded}")

```

1.1.5 三种分词方法对比

特性 BPE WordPiece SentencePiece

合并标准 频率最高 似然增益最大 支持两种模式

预处理需求 需要预分词 需要预分词 无需预分词

代表模型 GPT-2/3/4 BERT T5, ALBERT

多语言支持 一般 一般 优秀

词尾标记 可选 ##前缀 可配置

- - -

1.2 词向量的发展史

词向量是NLP中表征语义信息的核心方法，其发展经历了从稀疏到稠密、从静态到动态的演进。

1.2.1 One-hot 编码

原理：每个词对应词汇表中的一个独热向量，维度等于词汇表大小。

```

import numpy as np

vocab = ["猫", "狗", "鱼", "鸟"]
vocab_size = len(vocab)
word2idx = {w: i for i, w in enumerate(vocab)}

```

```
def onehot(word, vocabsiz, word2idx):
    vec = np.zeros(vocabsiz)
    vec[word2idx[word]] = 1
    return vec

print(onehot("猫", vocabsiz, word2idx)) # [1, 0, ...]
print(onehot("狗", vocabsiz, word2idx)) # [0, 1, ...]
```

致命缺陷：

- 维度灾难：词汇量10万则向量维度10万
- 语义缺失：任意两个词的内积为0，无法表达语义相似性
- 完全稀疏：只有一个位置为1，信息密度极低

1.2.2 Word2Vec 详解

Word2Vec 通过上下文预测来学习稠密词向量，有两种架构：

CBOW (Continuous Bag of Words)：用上下文词预测中心词

Skip-gram：用中心词预测上下文词

CBOW模型详解：

给定上下文词 $w_{t-m}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+m}$

1. 输入层：上下文词的 one-hot 向量
2. 隐藏层：上下文词向量的平均值
3. 输出层：Softmax 预测中心词

$$h = \frac{1}{2m} \sum_{i=-m, i \neq 0}^{m} v_{w_i}$$

$$P(w_t | \text{context}) = \frac{\exp(u_{w_t}^T h)}{\sum_{w \in V} \exp(u_w^T h)}$$

其中 v_w 是输入词向量矩阵 W_{in} 的行， u_w 是输出词向量矩阵 W_{out} 的列

Skip-gram模型详解：

给定中心词 w_t ，预测上下文词：

$$P(w_{t+j} | w_t) = \frac{\exp(u_{w_{t+j}}^T v_{w_t})}{\sum_{w'} \exp(u_{w'}^T v_{w_t})}$$

目标函数：

$$J = \frac{1}{T} \sum_{t=1}^T \sum_{j=1}^J -\log P(w_{t+j} | w_t)$$

Skip-gram的梯度推导：

以预测单个上下文词为例，记 w_O 为上下文词（输出）， w_I 为中心词（输入）：

$$\log P(w_O | w_I) = u_{w_O}^T v_{w_I} - \log \sum_{w=1}^V \exp(u_w^T v_{w_I})$$

对 v_{w_I} （输入向量）求偏导：

$$\frac{\partial \log P(w_O | w_I)}{\partial v_{w_I}} = u_{w_O} - \sum_{w=1}^V \frac{\exp(u_w^T v_{w_I})}{\sum_{w=1}^V \exp(u_w^T v_{w_I})} u_w$$

直觉解释：梯度 = 目标词向量 - 所有词向量的期望。如果模型已经正确预测，两者接近，梯度趋近于0。

对 u_w （输出向量）求偏导：

$$\frac{\partial \log P(w_O | w_I)}{\partial u_w} = \begin{cases} v_{w_I} & \text{if } w = w_O \\ -\frac{\exp(u_w^T v_{w_I})}{\sum_{w=1}^V \exp(u_w^T v_{w_I})} v_{w_I} & \text{if } w \neq w_O \end{cases}$$

负采样 (Negative Sampling)：

原始 Softmax 需要遍历整个词汇表，计算量巨大。负采样将多分类问题转化为二分类：

$$\log P(w_O | w_I) \approx \log \sigma(u_{w_O}^T v_{w_I}) - \sum_{w' \sim P_n} \log \sigma(-u_{w'}^T v_{w_I})$$

其中 σ 是 Sigmoid 函数， k 是负样本数量（通常 5 - 20）， $P_n(w)$ 是负样本分布：

$$P_n(w) = \frac{f(w)^{3/4}}{\sum_{w'} f(w')^{3/4}}$$

$3/4$ 次方的作用：略微提高低频词被采为负样本的概率，避免高频词主导训练。

Word2Vec 的完整实现：

```
import numpy as np
from collections import Counter
```

```

class SkipGram:
def __init__(self, vocabs_size, embeddingdim, window_size):
    self.vocabs_size = vocabs_size
    self.embeddingdim = embeddingdim
    self.window_size = window_size
    # 输入词向量矩阵 (Win)
    self.Win = np.random.randn(vocabs_size, embeddingdim)
    # 输出词向量矩阵 (Wout)
    self.Wout = np.random.randn(vocabs_size, embeddingdim)

def sigmoid(self, x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -500, 500)))

def trainpair(self, centerword, contextword, negsamples):
    """训练一个 (中心词, 上下文词) 对"""
    vin = self.Win[centerword] # 中心词向量

    # 正样本：上下文词
    gradin = np.zeros(self.embeddingdim)
    score = np.dot(self.Wout[contextword], vin)
    gradout = (self.sigmoid(score) - 1) * vin # 正样本梯度
    gradin += (self.sigmoid(score) - 1) * self.Wout[contextword]
    self.Wout[contextword] -= lr * gradout

    # 负样本
    for negword in negsamples:
        score = np.dot(self.Wout[negword], vin)
        gradout = self.sigmoid(score) * vin # 负样本梯度
        gradin += self.sigmoid(score) * self.Wout[negword]
        self.Wout[negword] -= lr * gradout

    # 更新中心词向量
    self.Win[centerword] -= lr * gradin

def getembedding(self, wordidx):

```

" " " 获取词向量 " " "

```
return self.Win[wordidx]
```

生成训练数据

```
def generatetrainingdata(tokens, windowsize):  
    " " " 从语料中生成 (中心词, 上下文词) 对 " " "  
    trainingpairs = []  
    for i, center in enumerate(tokens):  
        left = max(0, i - windowsize)  
        right = min(len(tokens), i + windowsize + 1)  
        for j in range(left, right):  
            if i != j:  
                trainingpairs.append((center, tokens[j]))  
    return trainingpairs
```

Word2Vec的重要性质——向量运算：

经典示例： $\text{king} - \text{man} + \text{woman} \approx \text{queen}$

```
result = model.getembedding('king') - model.getembedding('man') + model.getembedding('woman')
```

在词向量空间中寻找最近邻

```
mostsimilar = findmostsimilar(result, model)
```

期望结果：queen

这一性质说明Word2Vec的向量空间中存在线性关系，使得语义运算成为可能。

1.2.3 GloVe (Global Vectors)

GloVe 结合了全局统计信息和局部上下文窗口的优势。

核心思想：利用词共现矩阵的统计信息，通过加权最小二乘法学习词向量。

共现概率：设 X_{ij} 为词 i 和词 j 在语料中的共现次数， $X_i = \sum_j X_{ij}$ 与所有词的共现总次数，则共现概率 $P_{ij} = X_{ij} / X_i$ 。

关键洞察：共现概率的比值能区分语义关系：

比值 $P(k \text{ solid}) / P(k \text{ gas}) \approx P(k \text{ liquid}) / P(k \text{ solid})$

$k = \text{ice}$ 大 小 远大于1
 $k = \text{steam}$ 小 大 远小于1
 $k = \text{water}$ 中 中 接近1

目标函数：

$$J = \sum_{i=1}^V \sum_{j=1}^V f(X_{ij}) \left(X_{ij} - \text{right} \right)^2$$

其中权重函数 $f(x)$ 的设计：

$$f(x) = \begin{cases} (x / x_{\max})^{\alpha} & \text{if } x \leq x_{\max} \\ 0 & \text{otherwise} \end{cases}$$

- $x_{\max}$ 通常设为100， α 通常设为0.75
- 权重函数的作用：对高频共现进行截断，避免高频无意义词对（如 "the", "is"）主导损失

1.2.4 ELMo (Embeddings from Language Models)

ELMo是第一个广泛应用的上下文相关词向量方法。

核心思想：使用双向LSTM语言模型，为每个词生成依赖于上下文的动态词向量。

架构：

1. 前向LSTM：从左到右预测下一个词
2. 后向LSTM：从右到左预测上一个词
3. 多层BiLSTM堆叠

ELMo向量：对于第 k 个词，ELMo将所有层的隐状态加权组合：

$$\text{text}\{\text{ELMo}\}_k = \gamma \sum_{j=0}^L s_j h_{k,j}$$

其中 s_j 是可学习的softmax权重， γ 是缩放因子。

与静态词向量的本质区别：

- 静态词向量：每个词只有一种表示（"bank" 无论指银行还是河岸都一样）
- ELMo：同一个词在不同上下文中有不同表示（"bank" 在金融语境和自然语境中的向量不同）

1.2.5 从静态到动态：词向量发展总结

方法 类型 上下文感知 代表模型 年份

One-hot 稀疏 否 - -

Word2Vec 稠密 否 Skip-gram, CBOW 2013

GloVe 稠密 否 GloVe 2014

ELMo 稠密 是 BiLSTM 2018

BERT 稠密 是 Transformer 2018

- - -

二、文本分类的详细技术

2.1 传统方法详解

2.1.1 TF-IDF详解

TF (Term Frequency) —— 词频：

$$\text{TF}(t, d) = \frac{\text{词 } t \text{ 在文档 } d \text{ 中出现的次数}}{\text{文档 } d \text{ 中的总词数}}$$

IDF (Inverse Document Frequency) —— 逆文档频率：

$$\text{IDF}(t) = \log \frac{N + 1}{\text{DF}(t)}$$

其中 N 是文档总数， $\text{DF}(t)$ 是包含词 t 的文档数。加1是为了平滑。

TF-IDF：

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

直觉理解：一个词如果在某篇文档中出现很多次（TF高），同时在其他文档中很少出现（IDF高），则该词具有很强的区分能力。

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.naivebayes import MultinomialNB
from sklearn.pipeline import Pipeline
```

```
from sklearn.modelselection import train_test_split
from sklearn.metrics import classification_report
```

示例数据

```
texts = [
    "这部电影太精彩了，演员演技出色"，
    "剧情无聊，浪费时间的烂片"，
    "画面很美，但故事太单薄"，
    "强烈推荐，值得二刷的好电影"，
    "差评，特效粗糙剧情离谱"，
    # ... 更多数据
]
labels = [1, 0, 0, 1, 0] # 1=正面， 0=负面
```

划分训练集和测试集

```
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.2, random_state=42
)
```

==== 方法1：TF-IDF + SVM =====

```
svmpipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        max_features=10000, # 最多保留10000个特征
        ngram_range=(1, 2), # 使用unigram和bigram
        sublinear_tf=True, # 使用1+log(tf)替代tf，压缩高频词影响
        min_df=2, # 至少在2个文档中出现才保留
        max_df=0.95, # 在超过95%文档中出现的词被过滤
    )),
    ('svm', LinearSVC(C=1.0, max_iter=10000)),
])

svmpipeline.fit(X_train, y_train)
svmpred = svmpipeline.predict(X_test)
print("SVM分类报告：")
print(classification_report(y_test, svmpred))
```

===== 方法2： TF-IDF + 朴素贝叶斯 =====

```
nbpipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
maxfeatures=10000,
ngramrange=(1, 2),
)),
    ('nb', MultinomialNB(alpha=1.0)), # alpha是拉普拉斯平滑参数
])
```

```
nbpipeline.fit(Xtrain, ytrain)
nbpred = nbpipeline.predict(Xtest)
print("朴素贝叶斯分类报告：")
print(classificationreport(ytest, nbpred))
```

朴素贝叶斯分类器的数学原理：

朴素贝叶斯基于贝叶斯定理和条件独立性假设：

$$P(y | x_1, x_2, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i | y)$$

对于文本分类， x_i 是TF-IDF特征：

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^n P(x_i | y)$$

常见误区：

- 认为TF-IDF+朴素贝叶斯已经过时 → 在小数据集、低资源场景下仍是强baseline
- 忽略ngramrange参数 → 加入bigram/trigram能显著提升性能
- 不做文本预处理 → 去除停用词、统一大小写能提高特征质量

2.2 深度学习方法

2.2.1 TextCNN详解

TextCNN利用一维卷积操作捕捉文本中的局部n-gram特征。

架构：

1. 嵌入层：将词索引映射为词向量

2. 卷积层：使用多种尺寸的卷积核（如 2, 3, 4, 5）捕捉不同粒度的 n - gram
3. 池化层：对每个特征图进行最大池化，提取最重要的特征
4. 全连接层：分类输出

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class TextCNN(nn.Module):
    def __init__(self, vocabsize, embeddingdim, numclasses,
                 numfilters=100, filtersizes=[2, 3, 4, 5], dropout=0.5):
        super(TextCNN, self).__init__()
        # 嵌入层
        self.embedding = nn.Embedding(vocabsize, embeddingdim)
        # 多尺度卷积层：每种尺寸的卷积核各有numfilters个
        self.convs = nn.ModuleList([
            nn.Conv1d(
                inchannels=embeddingdim,
                outchannels=numfilters,
                kernelsize=fs
            )
            for fs in filtersizes
        ])
        # Dropout层
        self.dropout = nn.Dropout(dropout)
        # 全连接分类层
        # 输入维度 = 卷积核数量 × 卷积核尺寸种类数
        self.fc = nn.Linear(numfilters * len(filtersizes), numclasses)

    def forward(self, x):
        # x: [batchsize, seqlen]
        embedded = self.embedding(x) # [batchsize, seqlen, embeddingdim]
        embedded = embedded.permute(0, 2, 1) # [batchsize, embeddingdim, seqlen]

        # 对每种尺寸的卷积核分别卷积 + 最大池化
```

```

convoutputs = []
for conv in self.convs:
    c = F.relu(conv(embedded)) # [batchsize, numfilters, height, width]
    c = F.maxpool1d(c, c.size(2)) # [batchsize, numfilters, width]
    c = c.squeeze(2) # [batchsize, numfilters]
    convoutputs.append(c)

# 拼接所有卷积结果
cat = torch.cat(convoutputs, dim=1) # [batchsize, numfilters * width]
cat = self.dropout(cat)
output = self.fc(cat) # [batchsize, numclasses]
return output

```

TextCNN的设计直觉：

- 不同尺寸的卷积核分别捕捉bigram、trigram等不同粒度的语义
- 最大池化提取每个卷积核最显著的特征，实现位置不变性
- 类似于"扫描文本寻找关键模式"的过程

2.2.2 BiLSTM + Attention详解

BiLSTM捕捉双向序列特征，Attention机制赋予关键信息更高权重。

Attention机制：

$$e_t = \tanh(Wa_{ht} + ba)$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^T \exp(e_k)}$$

$$c = \sum_{t=1}^T \alpha_t h_t$$

其中 h_t 是BiLSTM在时刻 t 的隐状态， c 是上下文向量（加权求和）。

```

class BiLSTMAttention(nn.Module):
    def __init__(self, vocabsz, embeddingdim, hiddensz,
                  numlayers=2, dropout=0.5):
        super(BiLSTMAttention, self).__init__()
        self.embedding = nn.Embedding(vocabsz, embeddingdim)

```

```

self.lstm = nn.LSTM(
    inputsize=embeddingdim,
    hiddensize=hiddendim,
    numlayers=numlayers,
    batchfirst=True,
    bidirectional=True,
    dropout=dropout if numlayers > 1 else 0
)

# Attention层
self.attention = nn.Linear(hiddendim 2, 1)
self.dropout = nn.Dropout(dropout)
self.fc = nn.Linear(hiddendim 2, numclasses)

def forward(self, x):
    # x: [batchsize, seqlen]
    embedded = self.embedding(x) # [batchsize, seqlen, embeddingdim]

    lstmout, _ = self.lstm(embedded) # [batchsize, seqlen, hiddendim2]

    # 计算Attention权重
    attnweights = torch.softmax(
        self.attention(lstmout).squeeze(-1), # [batchsize, seqlen]
        dim=1
    )

    # 加权求和
    context = torch.bmm(
        attnweights.unsqueeze(1), # [batchsize, 1, seqlen]
        lstmout # [batchsize, seqlen, hiddendim2]
    ).squeeze(1) # [batchsize, hiddendim2]

    context = self.dropout(context)
    output = self.fc(context) # [batchsize, numclasses]
    return output, attnweights # 同时返回注意力权重，便于可视化分析

```

2.3 BERT微调文本分类

BERT微调是最主流的文本分类方法，在大多数任务上接近SOTA。

```
from transformers import BertTokenizer, BertForSequenceClassification
import torch
from torch.utils.data import Dataset
```

自定义数据集

```
class TextClassificationDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = self.texts[idx]
        label = self.labels[idx]
        encoding = self.tokenizer.encode_plus(
            text,
            max_length=self.max_length,
            padding='max_length',
            truncation=True,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'token_type_ids': encoding.get('token_type_ids', torch.zeros(
                self.max_length, dtype=torch.long
```

```
) ).flatten() ,  
'labels': torch.tensor(label, dtype=torch.long)  
}
```

加载模型和分词器

```
modelname = 'bert-base-chinese'  
tokenizer = BertTokenizer.from_pretrained(modelname)  
model = BertForSequenceClassification.from_pretrained(  
modelname,  
num_labels=2 # 二分类  
)
```

准备数据

```
traindataset = TextClassificationDataset(traintexts)  
evaldataset = TextClassificationDataset(evaltexts)
```

训练配置

```
trainingargs = TrainingArguments(  
outputdir='./results',  
numtrainepochs=3,  
perdevicetrainbatchsize=16,  
perdeviceevalbatchsize=32,  
warmupsteps=500, # 预热步数  
weightdecay=0.01, # 权重衰减  
learningrate=2e-5, # BERT推荐的学习率: 2e-5到5e-5  
evalstrategy='epoch',  
savestrategy='epoch',  
loadbestmodelatend=True,  
metricforbestmodel='accuracy',  
)
```

训练

```
trainer = Trainer(  
model=model,  
args=trainingargs,
```

```
traindataset=traindataset,
evaldataset=evaldataset,
)

trainer.train()
```

推理

```
def predict(text, model, tokenizer):
    inputs = tokenizer(text, return_tensors='pt', trunc
with torch.no_grad():
    outputs = model(inputs)
    probs = torch.softmax(outputs.logits, dim=-1)
    pred = torch.argmax(probs, dim=-1).item()
    return pred, probs[0].tolist()

result, confidence = predict("这部电影太好看了!", model,
print(f"预测标签: {result}, 置信度: {confidence}")
```

BERT 微调的关键细节：

- [CLS] token：BERT在句首添加特殊token [CLS]，其最终隐状态用作分类特征
- 学习率：推荐 $2e-5$ 到 $5e-5$ ，太大会破坏预训练权重
- Warmup：前10%的训练步数使用线性预热，从0逐渐增加到设定学习率
- 截断长度：通常设为128或256，根据任务调整

2.4 少样本学习和零样本分类

零样本分类

利用预训练模型的知识，无需任何标注数据即可分类：

```
from transformers import pipeline
```

零样本分类pipeline

```
classifier = pipeline("zero-shot-classification",

text = "这款手机的电池续航能力令人失望"
candidatelabels = ["产品评价", "技术讨论", "新闻资讯", "娱乐"]
```

```

result = classifier(text, candidatelabels)
print(f"文本: {text}")
for label, score in zip(result['labels'], result['scores']):
    print(f"{label}: {score:.4f}")

```

原理：零样本分类利用自然语言推理（NLI）模型，将分类问题转化为蕴含关系判断——“该文本是否蕴含‘本是关于XX的’这一假设”。

少样本学习（Prompt-based Learning）

通过设计提示模板，将分类任务转化为填空任务：

少样本学习示例：情感分类

传统方式：输入文本 → 分类器 → 正面 / 负面

提示方式：输入文本 + “这篇文章的情感是 [MASK]。” → BERT → 好 / 坏

```

from transformers import BertForMaskedLM, BertTokenizer

model = BertForMaskedLM.from_pretrained('bert-base-uncased')
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

def promptbasedclassification(text):
    # 设计提示模板
    prompt = f"这篇文章的情感是 [MASK] 的。内容: {text}"
    inputs = tokenizer(prompt, return_tensors='pt')
    mask_token_id = tokenizer.mask_token_id
    mask_pos = (inputs['input_ids'] == mask_token_id).nonzero().squeeze(-1)

    with torch.no_grad():
        outputs = model(inputs)
        logits = outputs.logits[0, mask_pos]

    # 定义正面词和负面词
    positivewords = ["好", "棒", "优秀", "精彩"]
    negativewords = ["差", "烂", "糟糕", "无聊"]

    pos_score = sum(logits[tokenizer.encode(w)[1]] for w in positivewords)
    neg_score = sum(logits[tokenizer.encode(w)[1]] for w in negativewords)

```

```

negscore = sum(logits[tokenizer.encode(w)[1]]) for
return "正面" if posscore > negscore else "负面"

- - -

```

三、命名实体识别（NER）的详细技术

3.1 BIO标注体系详解

BIO是最常用的序列标注方案：

标签 含义 示例

B - PER 人名开头 张 (三)
 I - PER 人名内部 张 (三)
 B - LOC 地名开头 北 (京)
 I - LOC 地名内部 北 (京)
 B - ORG 机构名开头 清 (华大学)
 I - ORG 机构名内部 清 (华大学)
 O 非实体 的、在、是等

完整标注示例：

原始文本： 张三在北京清华大学学习

BIO标注： B - PER I - PER O B - LOC I - LOC B - ORG I - ORG I - OR

BIO vs BIOES：

标注体系 实体开始 实体内部 实体结束 单字实体 非实体

BIO B - X I - X - B - X O
 BIOES B - X I - X E - X S - X O

BIOES比BIO更精细，明确标记了实体的结束位置，但BIO更简洁、更常用。

3.2 BiLSTM-CRF详解

BiLSTM-CRF是NER的经典深度学习模型，由三层组成：

架构：

1. 嵌入层：词向量 + 字符级向量（可选）
2. BiLSTM层：捕捉双向上下文特征
3. CRF层：建模标签之间的转移约束

为什么需要CRF层？

BiLSTM独立预测每个位置的标签，可能产生不合法的标注序列（如 I - P E R 出现在 O 后面）。CRF层通过学习标签转移概率来约束输出序列的合法性。

CRF的数学形式：

给定输入序列 $X = (x_1, \dots, x_n)$ ，标签序列 $Y = (y_1, \dots, y_n)$

$$s(X, Y) = \sum_{i=1}^n E\{y_i\} + \sum_{i=1}^{n-1} T\{y_i, y_{i+1}\}$$

其中 $E\{y_i\}$ 是BiLSTM在位置 i 对标签 y_i 的发射分数， $T\{y_i, y_{i+1}\}$ 的转移分数。

条件概率：

$$P(Y|X) = \frac{\exp(s(X, Y))}{\sum_{Y'} \exp(s(X, Y'))}$$

分母需要对所有可能的标签序列求和，使用前向算法高效计算。

训练目标：最大化对数似然：

$$\log P(Y|X) = s(X, Y) - \log \sum_{Y'} \exp(s(X, Y'))$$

解码：使用Viterbi算法找到得分最高的标签序列。

```
import torch
import torch.nn as nn

class BiLSTMCRF(nn.Module):
    def __init__(self, vocabsize, embeddingdim, hidddim):
        super().__init__()
        self.numtags = numtags
```

```

self.embedding = nn.Embedding(vocabsize, embeddingdim)
self.lstm = nn.LSTM(embeddingdim, hiddendim // 2,
numlayers=2, bidirectional=True, batchfirst=True)
self.hidden2tag = nn.Linear(hiddendim, numtags)

# CRF转移矩阵：T[i, j]表示从标签j转移到标签i的分数
# 注意：包含了开始和结束标签的转移
self.transitions = nn.Parameter(torch.randn(numtags, numtags))
# 不可能从任何标签转移到开始标签
self.transitions.data[:, 0] = -10000
# 不可能从结束标签转移到任何标签
self.transitions.data[1, :] = -10000

def getlstmfeatures(self, x):
    """BiLSTM前向传播，获取发射分数"""
    embeds = self.embedding(x)
    lstmout, _ = self.lstm(embeds)
    lstmfeats = self.hidden2tag(lstmout)
    return lstmfeats

def forwardalg(self, feats):
    """前向算法：计算所有路径的分数之和(log-sum-exp)"""
    initalphas = torch.full((1, self.numtags), -10000)
    initalphas[0][0] = 0. # 开始标签的分数为0

    forwardvar = initalphas
    for feat in feats:
        alphas = []
        for nexttag in range(self.numtags):
            # 发射分数 + 转移分数
            emitscore = feat[nexttag].view(1, -1).expand(1, self.numtags)
            transscore = self.transitions[nexttag].view(1, -1)
            nexttagvar = forwardvar + transscore + emitscore
            alphas.append(torch.logsumexp(nexttagvar, dim=1))
        forwardvar = torch.cat(alphas).view(1, -1)

```

```

terminalvar = forwardvar + self.transitions[1] #
alpha = torch.logsumexp(terminalvar, dim=1)
return alpha

def scoresentence(self, feats, tags):
    """ 计算给定标签序列的分数 """
    score = torch.zeros(1)
    tags = torch.cat([torch.tensor([0]), tags]) # 开头加
    for i, feat in enumerate(feats):
        score = score + self.transitions[tags[i+1], tags[i]]
        score = score + self.transitions[1, tags[-1]] # 结尾
    return score

def negloglikelihood(self, x, tags):
    """ 负对数似然损失 """
    feats = self.getlstmfeatures(x)
    forwardscore = self.forwardalg(feats)
    goldscore = self.scoresentence(feats, tags)
    return forwardscore - goldscore

def forward(self, x):
    """ 推理时使用Viterbi解码 """
    feats = self.getlstmfeatures(x)
    # Viterbi解码 (此处省略详细实现, 可使用pytorch-crf库)
    bestpath = self.viterbidecode(feats)
    return bestpath

```

3.3 BERT-CRF

将BiLSTM替换为BERT, 通常能获得更好的性能:

```

from transformers import BertModel

class BertCRF(nn.Module):
    def __init__(self, bertmodelname, numtags, dropout=0.1):

```

```

super().init()
self.bert = BertModel.from_pretrained(bert_model_name)
self.dropout = nn.Dropout(dropout)
self.classifier = nn.Linear(self.bert.config.hidden_size, numtags)
self.crf = CRF(numtags, batch_first=True) # 使用pytorch

def forward(self, input_ids, attention_mask, labels):
    bert_output = self.bert(input_ids=input_ids, attention_mask=attention_mask)
    sequence_output = bert_output.last_hidden_state
    sequence_output = self.dropout(sequence_output)
    logits = self.classifier(sequence_output)

    if labels is not None:
        # 训练时计算CRF损失
        loss = -self.crf(logits, labels, mask=attention_mask)
        return loss
    else:
        # 推理时使用CRF解码
        prediction = self.crf.decode(logits, mask=attention_mask)
        return prediction

```

3.4 实体链接和关系抽取

实体链接 (Entity Linking)

实体链接将文本中的实体指称 (mention) 映射到知识库中的实体：

文本： " 苹果发布了新 iPhone "

指称： " 苹果 " → 知识库实体： Apple Inc. (而非水果)

方法：

1. 候选生成：根据指称文本从知识库中检索候选实体
2. 实体消歧：利用上下文信息从候选中选择正确实体

关系抽取 (Relation Extraction)

识别实体之间的关系：

文本： " 乔布斯创立了苹果公司 "

实体： 乔布斯 (P E R) , 苹果公司 (O R G)

关系： 创始人 (F o u n d e r o f)

方法分类：

方法 思路 优缺点

基于模板 匹配预定义的模式 高精确率，低召回率

基于特征 手工设计特征 + 分类器 依赖特征工程

基于深度学习 端到端学习 自动学特征，但需大量数据

基于远程监督 利用知识库自动标注 噪声多，需去噪

B E R T 关系抽取：

```
class BertForRelationExtraction(nn.Module):
    def __init__(self, bertmodelname, numrelations):
        super().__init__()
        self.bert = BertModel.from_pretrained(bertmodelname)
        self.hiddensize = self.bert.config.hiddensize

        # 实体表示：用实体标记的特殊token的隐状态
        self.classifier = nn.Sequential(
            nn.Linear(hiddensize, hiddensize),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(hiddensize, numrelations)
        )

    def forward(self, inputids, attentionmask, e1pos,
                e2pos):
        """
        e1pos, e2pos: 实体在序列中的位置
        """
        outputs = self.bert(inputids=inputids, attentionmask=attentionmask)
        hiddenstates = outputs.last_hidden_state # [batch,
```

```

# 提取两个实体的表示
batchsize = hiddenstates.size(0)
e1repr = hiddenstates[torch.arange(batchsize), e1]
e2repr = hiddenstates[torch.arange(batchsize), e2]

# 拼接两个实体的表示
combined = torch.cat([e1repr, e2repr], dim=-1) #
logits = self.classifier(combined)
return logits

- - -

```

四、机器翻译的详细技术

4.1 Seq2Seq + Attention详解

编码器 - 解码器架构

编码器：将源语言句子编码为固定长度的上下文向量

解码器：根据上下文向量逐词生成目标语言句子

注意力机制解决了固定长度瓶颈问题：

$$e_{ij} = \text{text_score}(h_i, s_j)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{Tx} \exp(e_{ik})}$$

$$c_i = \sum_{j=1}^{Tx} \alpha_{ij} h_j$$

其中 h_j 是编码器隐状态， s_j 是解码器隐状态， c_i 是上下文向量。

打分函数类型：

类型 公式 特点

加性 (Bahdanau) $v^T \tanh(W1 \cdot h + W2 \cdot s)$ 参数多，表达力强

乘性 (Luong) $h^T W s$ 计算高效

点积 $h^T s$ 最简单，无需额外参数

缩放点积 $\frac{h^T s}{\sqrt{d}}$ 防止点积值过大

4.2 Transformer 翻译模型

Transformer 用自注意力机制替代 RNN，实现并行计算：

关键组件：

- 多头注意力：多个注意力头并行计算，捕捉不同子空间的语义
- 位置编码：注入位置信息，弥补无递归结构的位置感知缺失
- 前馈网络：对每个位置独立应用非线性变换

编码器自注意力：源语言内部词与词之间的关联

解码器自注意力：目标语言内部词与词之间的关联（带掩码，防止看到未来）

交叉注意力：解码器关注编码器的输出

4.3 BLEU 评估指标详解

BLEU (Bilingual Evaluation Understudy) 是机器翻译最常用的自动评估指标

核心思想：衡量机器翻译与参考翻译之间的 n -gram 重合度。

计算步骤：

1. 计算修正 n -gram 精确率：

$$p_n = \frac{\sum_{c \in C} \min(\text{Count}\{\text{clip}\}(n\text{-gram}), \text{Count}(n\text{-gram}))}{\sum_{c \in C} \text{Count}(n\text{-gram})}$$

其中 $\text{Count}\{\text{clip}\}$ 是截断计数，限制每个 n -gram 最多出现次数不超过参考翻译中的最大次数。

截断计数示例：

候选： the the the the the

参考： the cat sat on the mat

"the" 在候选中出现 5 次，在参考中出现 2 次

截断后 "the" 的贡献为 $\min(5, 2) = 2$

1 - gram精确率 = $2 / 5 = 0.4$ (而非 $5 / 5 = 1.0$)

2. 计算简洁惩罚 (Brevity Penalty) :

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{1 - r/c} & \text{otherwise} \end{cases}$$

其中 c 是候选翻译长度, r 是参考翻译长度。BP惩罚过短的翻译。

3. 最终BLEU分数:

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^N w_n\right)$$

通常 $N=4$, $w_n = 1/N$, 即等权重几何平均。

```
from collections import Counter
import math
```

```
def computebleu(candidate, references, maxn=4):
    """
```

计算BLEU分数

candidate: 候选翻译的词列表

references: 参考翻译列表, 每个是词列表

"""

```
# 计算修正n-gram精确率
```

```
precisions = []
```

```
for n in range(1, maxn + 1):
```

```
# 提取候选的n-grams
```

```
candidatengrams = Counter(
```

```
tuple(candidate[i:i+n]) for i in range(len(candidate) - n + 1))
```

```
if not candidatengrams:
```

```
precisions.append(0)
```

```
continue
```

```
# 对每个n-gram, 取参考翻译中的最大出现次数作为上限
```

```
clippedcount = 0
```

```
totalcount = 0
```

```

for ngram, count in candidatengrams.items():
    maxrefcount = max(
        Counter(tuple(ref[i:i+n]) for i in range(len(ref)
        for ref in references
    ))
    clippedcount += min(count, maxrefcount)
    totalcount += count

precisions.append(clippedcount / totalcount)

# 简洁惩罚
c = len(candidate)
r = min(references, key=lambda ref: abs(len(ref)
r = len(r)

if c > r:
    bp = 1
elif c == 0:
    bp = 0
else:
    bp = math.exp(1 - r / c)

# 几何平均
if min(precisions) == 0:
    return 0
logavg = sum(math.log(p) for p in precisions) / n
bleu = bp * math.exp(logavg)
return bleu

- - -

```

五、文本生成的详细技术

5.1 解码策略详解

给定语言模型的概率分布 $P(w_t | w_{<t})$ ，如何选择下一个 token？

5.1.1 贪心解码 (Greedy Decoding)

每步选择概率最高的 token:

$$w_t = \arg\max_w P(w | w_{<t})$$

问题: 容易陷入局部最优, 错过全局最优序列。

5.1.2 束搜索 (Beam Search)

维护 k 个候选序列 (beam size = k), 每步扩展所有候选并保留得分最高的 k 个

```
def beamsearch(model, initialinput, beamwidth=5,
               "
```

Beam Search 解码

lengthpenalty: 长度惩罚, > 0 鼓励更长的序列

```
"
```

```
# 初始化: 每个 beam 存储 (序列, 对数概率)
```

```
beams = [(initialinput, 0.0)]
```

```
for step in range(maxlength):
```

```
    newbeams = []
```

```
    for sequence, score in beams:
```

```
        # 如果序列已经结束, 直接加入新候选
```

```
        if sequence[-1] == '<EOS>':
```

```
            newbeams.append((sequence, score))
```

```
            continue
```

```
        # 获取模型预测
```

```
        probs = model.predictnext(sequence)
```

```
        logprobs = torch.log(probs)
```

```
        # 扩展 beamwidth 个最佳候选
```

```
        topprobs, topindices = logprobs.topk(beamwidth)
```

```
        for i in range(beamwidth):
```

```
            newsequence = sequence + [topindices[i].item()]
```

```

# 长度惩罚：归一化分数
newscore = score + topprobs[i].item()
newbeams.append((newsequence, newscore))

# 按得分排序，保留top-k（应用长度惩罚）
newbeams.sort(
    key=lambda x: x[1] / (len(x[0]) * lengthpenalty),
    reverse=True
)
beams = newbeams[:beamwidth]

return beams[0][0] # 返回得分最高的序列

```

5.1.3 Top-k采样

从概率最高的 k 个token中按概率采样：

$$P'(w_t) = \frac{P(w_t | w_{<t})}{\sum_{w \in V_k} P(w_t | w_{<t})}$$

其中 V_k 是概率最高的 k 个token的集合。

```

def topksampling(logits, k=50, temperature=1.0):
    """
    Top-k采样
    logits: 模型输出的logits [vocabsize]
    k: 只保留概率最高的k个token
    temperature: 温度参数
    """
    # 温度缩放
    logits = logits / temperature

    # 只保留top-k
    topklogits, topkindices = torch.topk(logits, k)
    # 将其余位置设为负无穷
    filteredlogits = torch.full_like(logits, float('-inf'))
    filteredlogits.scatter(0, topkindices, topklogits)

```

```
# 采样
probs = torch.softmax(filtered_logits, dim=-1)
next_token = torch.multinomial(probs, num_samples=1)
return next_token
```

5.1.4 Top-p (Nucleus) 采样

选择累积概率刚好超过 p 的最小 token 集合：

```
def topp_sampling(logits, p=0.9, temperature=1.0):
    """
    Top-p (Nucleus) 采样
    p: 累积概率阈值
    """
    logits = logits / temperature
    sorted_logits, sorted_indices = torch.sort(logits,
        sorted_probs = torch.softmax(sorted_logits, dim=-1)
    cumulative_probs = torch.cumsum(sorted_probs, dim=-1)

    # 移除累积概率超过p的 token (保留第一个超过p的)
    sorted_mask = cumulative_probs - sorted_probs > p
    sorted_logits[sorted_mask] = float('-inf')

    # 恢复原始顺序
    filtered_logits = torch.zeros_like(logits)
    filtered_logits.scatter(0, sorted_indices, sorted_logits)

    probs = torch.softmax(filtered_logits, dim=-1)
    next_token = torch.multinomial(probs, num_samples=1)
    return next_token
```

5.1.5 Temperature 参数

Temperature 控制输出分布的 " 尖锐程度 "：

$$P'(w_t) = \frac{\exp(\logit\{w_t\} / T)}{\sum_w \exp(\logit\{w\} / T)}$$

- $T \rightarrow 0$: 分布趋于 one-hot (趋向贪心)
- $T = 1$: 原始分布
- $T \rightarrow \infty$: 分布趋于均匀 (完全随机)

各策略对比：

策略 多样性 质量 适用场景

Greedy 最低 可能陷入局部最优 翻译、摘要

Beam Search 低 高 翻译、摘要

Top-k 中 中高 故事生成、对话

Top-p 中高 中高 通用文本生成

High Temperature 高 低 创意写作、头脑风暴

5.2 摘要生成

抽取式摘要

直接从原文中选择关键句子组成摘要：

```
from sklearn.metrics.pairwise import cosinesimilarity
import numpy as np

def extractivesummary(textsentences, numsentences):
    """基于TextRank的抽取式摘要"""
    # 1. 计算句子向量(可以用BERT或TF-IDF)
    vectorizer = TfidfVectorizer()
    sentencevectors = vectorizer.fit_transform(textsentences)

    # 2. 构建相似度矩阵
    similaritymatrix = cosinesimilarity(sentencevectors)

    # 3. 用PageRank思想迭代计算句子重要性
    scores = np.ones(len(textsentences)) / len(textsentences)
    d = 0.85 # 阻尼系数

    for i in range(50): # 迭代50次
```

```

newscores = np.zeroslike(scores)
for i in range(len(textsentences)):
    for j in range(len(textsentences)):
        if i != j and similaritymatrix[j].sum() > 0:
            newscores[i] += similaritymatrix[j][i] / similaritymatrix[j].sum()
newscores[i] = (1 - d) / len(textsentences) + d * newscores[i]
scores = newscores

# 4. 选择得分最高的句子
topindices = np.argsort(scores)[-numsentences:]
topindices = sorted(topindices) # 保持原文顺序
summary = [textsentences[i] for i in topindices]
return '。'.join(summary)

```

生成式摘要

使用序列到序列模型生成新的摘要文本，而非直接抽取。

BART和PEGASUS是专门为摘要设计的预训练模型：

- BART：去噪自编码器，预训练时对文本加噪再恢复
- PEGASUS：预训练时直接练习 " 句子级掩码恢复 "，天然适合摘要

5.3 对话系统设计

任务型对话系统

任务型对话系统通常包含以下模块：

用户输入

- 自然语言理解（NLU）：意图识别 + 槽位提取
- 对话状态跟踪（DST）：维护当前对话状态
- 对话策略（DPL）：决定下一步动作
- 自然语言生成（NLG）：生成回复

简化的任务型对话系统

```

class TaskOrientedDialog:
    def init(self):

```

```

self.state = {
    'intent': None,
    'slots': {}, # 槽位字典, 如 {'出发地': '北京', '目的地': '上海'}
    'history': []
}

def nlu(self, userinput):
    """自然语言理解: 意图识别 + 槽位提取"""
    intent = self.detectintent(userinput)
    slots = self.extractslots(userinput)
    return intent, slots

def dst(self, intent, slots):
    """对话状态跟踪: 更新当前状态"""
    self.state['intent'] = intent or self.state['intent']
    self.state['slots'].update({k: v for k, v in slots.items()})

    # 检查缺失的必要槽位
    requiredslots = self.getrequiredslots(self.state['intent'])
    missing = [s for s in requiredslots if s not in self.state['slots']]
    return missing

def dpl(self, missingslots):
    """对话策略: 决定下一步动作"""
    if missingslots:
        return f"ask{missingslots[0]}" # 询问缺失槽位
    else:
        return "execute" # 执行任务

def nlg(self, action):
    """自然语言生成"""
    templates = {
        'ask出发地': "请问您从哪里出发?",
        'ask目的地': "请问您要去哪里?",
        'ask日期': "请问您哪天出发?",
    }

```

```

'execute': f"好的,已为您查询从{self.state['slots'].get('出发地')}到{self.state['slots'].get('目的地')}的航班。"
}
return templates.get(action, "抱歉,我不太理解。")

- - -

```

六、问答系统的详细设计

6.1 机器阅读理解 (MRC)

给定文章和问题,从文章中抽取答案。

SQuAD数据集:最著名的MRC数据集,答案是文章中的连续片段。

方法演进:

1. BiDAF (2017): 双向注意力流
2. BERT (2018): 预训练模型直接微调
3. UnifiedQA (2020): 统一多种QA格式

BERT抽取式MRC:

预测答案的起始和结束位置:

$$P\{\text{start}\}(i) = \text{softmax}(W_s \cdot h_i)$$

$$P\{\text{end}\}(i) = \text{softmax}(W_e \cdot h_i)$$

其中 h_i 是BERT第 i 个token的隐状态。

```

from transformers import BertForQuestionAnswering

model = BertForQuestionAnswering.from_pretrained('bert-base-uncased')

def extract_answer(question, context, model, tokenizer):
    inputs = tokenizer(question, context, return_tensors='pt',
truncation=True, max_length=512)
    with torch.no_grad():
        outputs = model(inputs)

```

```

start_logits = outputs.start_logits[0]
end_logits = outputs.end_logits[0]

# 找到最佳答案范围
start_idx = torch.argmax(start_logits).item()
end_idx = torch.argmax(end_logits).item()

# 确保end >= start
if end_idx < start_idx:
    end_idx = start_idx

# 解码答案
input_ids = inputs['input_ids'][0]
answer_ids = input_ids[start_idx:end_idx+1]
answer = tokenizer.decode(answer_ids, skip_special_tokens=True)
return answer

```

6.2 知识图谱问答 (KBQA)

利用结构化知识图谱回答问题：

问题："姚明的妻子是谁？"

- 实体识别： 姚明
- 关系识别： 妻子
- 知识图谱查询： (姚明, 配偶, ?)
- 答案： 叶莉

方法分类：

- 语义解析 (SP)：将自然语言问题转化为 SPARQL 等查询语言
- 信息检索 (IR)：从知识图谱子图中检索答案

6.3 RAG 问答

RAG (Retrieval - Augmented Generation) 结合检索和生成：

流程：

1. 检索：从文档库中检索与问题相关的文档片段
2. 增强：将检索到的文档作为上下文拼接到问题中
3. 生成：语言模型基于增强后的输入生成答案

```
from transformers import AutoTokenizer, AutoModel
from sentence transformers import SentenceTransformer
import faiss

class RAGSystem:
    def __init__(self, documents, encoder_model='paraphrase-multilingual-MiniLM-L12-v2',
                  generator_model='bigscience/bloom-560m'):
        # 文档编码器
        self.encoder = SentenceTransformer(encoder_model)
        # 生成模型
        self.gentokenizer = AutoTokenizer.from_pretrained(generator_model)
        self.generator = AutoModelForCausalLM.from_pretrained(generator_model)

        # 构建 FAISS 索引
        self.documents = documents
        docembeddings = self.encoder.encode(documents)
        dimension = docembeddings.shape[1]
        self.index = faiss.IndexFlatIP(dimension) # 内积索引
        faiss.normalizeL2(docembeddings) # 归一化后内积 = 余弦相似度
        self.index.add(docembeddings)

    def retrieve(self, query, topk=3):
        """ 检索相关文档 """
        queryembedding = self.encoder.encode([query])
        faiss.normalizeL2(queryembedding)
        scores, indices = self.index.search(queryembedding, topk)
        return [self.documents[i] for i in indices[0]]

    def generate(self, query, retrieved_docs):
        """ 基于检索文档生成答案 """
        context = "\n".join(retrieved_docs)
```

```

prompt = f " 根据以下信息回答问题：\n\n{ context }\n\n问题：{ query } "
inputs = self.gentokenizer(prompt, return_tensors="pt")
outputs = self.generator.generate(
    inputs, max_new_tokens=200, temperature=0.7, do_sample=True)
answer = self.gentokenizer.decode(outputs[0], skip_special_tokens=True)
return answer

def answer(self, query):
    """完整RAG流程"""
    docs = self.retrieve(query)
    return self.generate(query, docs)

- - -

```

七、NLP中常用的预训练模型详解

7.1 架构对比

模型 架构 预训练任务 参数量 适用任务

BERT	编码器	MLM + NSP	110M / 340M	理解类（分类、NER、QA）
GPT-2 / 3	解码器	CLM（因果语言模型）	117M ~ 175B	生成类（对话、续写）
T5	编码器 - 解码器	Span Corruption	220M ~ 11B	理解 + 生成
BART	编码器 - 解码器	去噪自编码	140M / 400M	生成类（摘要、翻译）

7.2 预训练任务详解

BERT的预训练任务

MLM (Masked Language Model) :

- 随机遮盖15%的 token
- 被遮盖的 token 中：80%替换为 [MASK]，10%替换为随机词，10%保持不变
- 模型预测被遮盖的原始 token

为什么不是100%替换为 [MASK] ?

因为 [MASK] 在微调时不会出现，如果只做替换，模型会学到 " 看到 [MASK] 就预测 " 的捷径，而非真正理解语义。

NSP (Next Sentence Prediction) :

- 输入句子对 (A, B)
- 50%的B确实是A的下一句 (正样本)
- 50%的B是随机抽取的句子 (负样本)
- 模型判断B是否是A的下一句

NSP的争议: RoBERTa的研究表明NSP对下游任务帮助不大, 甚至有害。后续模型 (ALBERT) 做了改进。

GPT的预训练任务

CLM (Causal Language Model) : 标准语言模型, 从左到右预测下一个词:

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_1, \dots, w_{i-1})$$

GPT不做双向预测, 但更大的数据和模型规模弥补了这一 "不足"。

T5的预训练任务

Span Corruption: 随机遮盖文本片段, 用特殊哨兵 token 替代, 模型恢复原始文本:

输入: "Thank you <X> me to your party <Y> week"

目标: "<X> for inviting <Y> last <Z>"

T5将所有NLP任务统一为 "文本到文本" 格式。

7.3 微调策略

全参数微调

更新模型的所有参数, 效果最好但计算成本最高。

参数高效微调 (PEFT)

只更新少量参数, 大幅降低计算成本:

LoRA (Low-Rank Adaptation) :

将权重更新分解为低秩矩阵:

$$W' = W + \Delta W = W + BA$$

其中 $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times v}$

```
from peft import LoraConfig, get_peft_model
```

配置LoRA

```
loraconfig = LoraConfig(
    r=8, # 低秩矩阵的秩
    lora_alpha=16, # 缩放因子
    target_modules=["query", "value"], # 只对Q和V矩阵加LoRA
    lora_dropout=0.1,
    task_type="SEQCLS"
)
```

应用LoRA

```
model = BertForSequenceClassification.from_pretrained(
    model_name, num_labels=num_labels)
model = get_peft_model(model, loraconfig)
model.print_trainable_parameters()
输出: trainable params: 294,914 all params: 109,482,112
```

其他PEFT方法：

方法 思路 可训练参数占比

LoRA 低秩矩阵分解 ~ 0.3%

Prefix Tuning 在每层添加可学习前缀 ~ 0.1%

Prompt Tuning 仅学习软提示 ~ 0.01%

Adapter 在Transformer层间插入小网络 ~ 1%

八、中文NLP的特殊挑战

8.1 中文分词

中文没有天然的分隔符（如英文的空格），分词是中文NLP的基础但也是难点：

```
import jieba
```

```
text = "自然语言处理是人工智能的重要方向"
```

```
words = jieba.lcut(text)
```

```
print(words) # ['自然语言', '处理', '是', '人工智能', '的', '']
```

分词的歧义问题：

"结婚的和尚未结婚的"

→ 错误： 结婚 / 的 / 和尚 / 未 / 结婚 / 的

→ 正确： 结婚 / 的 / 和 / 尚未 / 结婚 / 的

解决方案：现代中文NLP趋向于使用字符级别或子词级别，避免分词歧义。BERT-base-Chinese的。

8.2 中文实体识别的挑战

- 嵌套实体："北京大学人民医院"既是ORG又是FAC
- 简称和别名："北大" = "北京大学"
- 领域术语：医疗、法律等领域有大量专业术语

8.3 中文语义理解

- 零代词（空主语）：中文常省略主语，共指消解困难
- 语序灵活：中文语序相对自由，句法分析更复杂
- 量词和修饰语："三个苹果" vs "三斤苹果"的语义差异

- - -

九、HuggingFace Transformers库的详细使用教程

9.1 基本使用

```
from transformers import AutoTokenizer, AutoModel
```

自动加载模型和分词器

```
modelname = "bert-base-chinese"
```

```
tokenizer = AutoTokenizer.from_pretrained(modelname)
```

```
model = AutoModel.from_pretrained(modelname)
```

文本编码

```
text = "自然语言处理"
inputs = tokenizer(text, return_tensors='pt')
print(f"Input IDs: {inputs['input_ids']}")
print(f"Attention Mask: {inputs['attention_mask']}")
print(f"Token Type IDs: {inputs['token_type_ids']}")
print(f"解码: {tokenizer.decode(inputs['input_ids'])}")
```

模型前向传播

```
outputs = model(inputs)
print(f"最后隐状态形状: {outputs.last_hidden_state.shape}")
```

9.2 Pipeline快速使用

```
from transformers import pipeline
```

文本分类

```
classifier = pipeline("text-classification",
model="uer/roberta-base-finetuned-jd-binary-chinese")
result = classifier("这个产品非常好用")
print(result) # [{'label': 'positive', 'score': 0.9999999999999999}]
```

命名实体识别

```
ner = pipeline("ner", model="uer/roberta-base-finetuned-ner-chinese")
entities = ner("张三在北京清华大学学习")
for ent in entities:
    print(f"{ent['word']}: {ent['entity']} ({ent['score']})")
```

问答

```
qa = pipeline("question-answering", model="uer/roberta-base-finetuned-qa-chinese")
answer = qa(question="中国的首都是哪里?", context="北京是中国的首都")
print(f"答案: {answer['answer']}")
```

文本生成

```
generator = pipeline("text-generation", model="uer/roberta-base-finetuned-gen-chinese")
```

```
result = generator("人工智能的未来", maxlength=50, numre
print(result[0]['generatedtext'])
```

9.3 自定义数据集微调

```
from transformers import (
AutoTokenizer, AutoModelForTokenClassification,
TrainingArguments, Trainer, DataCollatorForTokenC
)
from datasets import Dataset
```

准备NER数据

```
def tokenizeandalignlabels(examples):
"""将分词结果与标签对齐"""
tokenizedinputs = tokenizer(
examples["tokens"], truncation=True, issplitintov
)
labels = []
for i, label in enumerate(examples["nertags"]):
wordids = tokenizedinputs.wordids(batchindex=i)
previouswordidx = None
labelids = []
for wordidx in wordids:
if wordidx is None:
labelids.append(-100) # 特殊token的标签设为-100, 不参与损失计算
elif wordidx != previouswordidx:
labelids.append(label[wordidx]) # 第一个子词使用真实标签
else:
labelids.append(label[wordidx]) # 后续子词也使用相同标签
previouswordidx = wordidx
labels.append(labelids)
tokenizedinputs["labels"] = labels
return tokenizedinputs
```

加载模型

```
model = AutoModelForTokenClassification.from_pretrained(
    "bert-base-chinese", num_labels=9 # BIOESX标签数量
)
```

训练配置

```
trainingargs = TrainingArguments(
    output_dir="/nermodel",
    evaluation_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    warmup_ratio=0.1,
)
```

训练

```
trainer = Trainer(
    model=model,
    args=trainingargs,
    train_dataset=tokenized_train_dataset,
    eval_dataset=tokenized_eval_dataset,
    data_collator=DataCollatorForTokenClassification(tokenizer)
)

trainer.train()
model.save_pretrained("/nermodel")
tokenizer.save_pretrained("/nermodel")
```

9.4 模型选择指南

任务 推荐模型 理由

文本分类 BERT / RoBERTa 理解能力强

NER BERT-CRF 序列标注经典

问答 BERT / DPR 抽取式 / 检索式
摘要 BART / PEGASUS 专为摘要设计
翻译 mBART / mT5 多语言支持
对话 GPT / ChatGLM 生成能力强
通用 T5 文本到文本统一框架

- - -

十、NLP 核心评测基准详解

10.1 理解类基准

基准 任务类型 评估能力

GLUE 9项NLU任务 通用语言理解
SuperGLUE 更难的NLU任务 推理、共指等高级能力
SQuAD 2.0 阅读理解 含无答案问题
MNLI 自然语言推理 文本蕴含判断
CoNLL-2003 NER 实体识别标准

10.2 生成类指标

BLEU (见4.3节详解)

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

侧重召回率，常用于摘要评估：

$$\text{ROUGE-N} = \frac{\sum_{S \in \text{Ref}} \text{Count}(\text{match})_{\text{n-gram}}}{\sum_{S \in \text{Count}} \text{Count}(\text{n-gram})}$$

指标 含义 适用场景

ROUGE-1 unigram召回率 基本词汇匹配
ROUGE-2 bigram召回率 流畅性评估
ROUGE-L 最长公共子序列 结构匹配

BERTScore：

利用 B E R T 计算语义相似度，而非简单的 n - g r a m 匹配：

$$\text{BERTScore} = \frac{1}{|x|} \sum_{i \in x}$$

优势：能识别语义等价但词汇不同的表达（如 " 高兴 " ≈ " 开心 "）。

- - -

十一、未来趋势与挑战

- 1 . 多模态 N L P：结合文本、图像、语音的跨模态理解（如视觉问答、图文互生成）
- 2 . 低资源 N L P：通过少样本学习、跨语言迁移解决小语种问题
- 3 . 可解释性：提升模型决策的可解释性，满足医疗、金融等高风险领域需求
- 4 . 伦理与偏见：检测并消除模型中的性别、种族等偏见
- 5 . 长文本理解：超长上下文的 N L P 任务（长文档摘要、长对话理解）
- 6 . 多语言统一模型：一个模型覆盖多语言，促进低资源语言发展
- 7 . 检索增强生成：R A G 成为标准范式，结合外部知识提升生成质量
- 8 . A g e n t 化 N L P：L L M 作为 N L P 系统的核心推理引擎，结合工具使用

- - -

核心总结*：N L P 的任务从底层的分词、词性标注，到中层的关系抽取，再到高层的问答、翻译、生成了从 " 理解语言 " 到 " 生成语言 " 的完整技术栈。B E R T 和 G P T 的出现分别推动了理解类和生成类任务的，前，大语言模型正在统一 N L P 的各种任务，但传统方法（T F - I D F、C R F 等）在小数据、高精度要求的不可替代的价值。掌握从基础到前沿的完整知识体系，才能在实际问题中选择最合适的技术方案。